

Pronalaženje praznog pravougaonika maksimalne površine

Darija Eremija

Sadržaj

1 Problem koji se rešava	2
2 Naivno rešenje	2
3 Predloženo rešenje	2
3.1 Ispitivanje validnosti pravougaonika	2
4 Složenost	3
4.1 Vremenska složenost	3
4.2 Prostorna složenost	3
5 Implementacija	4
5.1 Primeri rada programa	4

1 Problem koji se rešava

U ovom projektu je za dati skup tačaka u ravni sa pozitivnim celobrojnim koordinatama potrebno odrediti pravougaonik najveće površine čije su stranice paralelne koordinatnim osama, a temena pripadaju zadatom skupu.

Pravougaonik se smatra validnim ako se na njegovim ivicama ili u unutrašnjosti ne nalazi nijedna tačka osim samih temena.

2 Naivno rešenje

Najjednostavniji način rešavanja ovog problema je provera svih potencijalnih pravougaonika direktnom pretragom. Pravougaonik čije su stranice paralelne koordinatnim osama je jednoznačno određen parom naspramnih temena (x_1, y_1) i (x_2, y_2) , pri čemu postoji $\mathcal{O}(N^2)$ takvih parova.

Za svaki par se najpre proverava da li su preostala dva temena u skupu tačaka. Ukoliko sva četiri temena postoje, potrebno je proveriti da li je pravougaonik prazan. Provera se vrši linearnim prolaskom kroz preostale tačke i direktnim ispitivanjem da li se neka od njih nalazi u pravougaoniku ili na njegovim ivicama.

Pošto je provera svakog kandidata složenosti $\mathcal{O}(N)$, ukupna vremenska složenost ovog algoritma u najgorem slučaju iznosi $\mathcal{O}(N^3)$.

3 Predloženo rešenje

Umesto linearne pretrage, algoritam kombinuje tehniku brišuće prave (*sweep-line*) i perzistentno segmentno stablo, čime se provera kandidata izvršava u logaritamskom vremenu.

Algoritam se sastoji iz sledećih koraka:

1. **Kompresija koordinata:** Kako koordinate tačaka mogu imati proizvoljno velike vrednosti, vrši se kompresija X koordinata. Sve jedinstvene X koordinate se sortiraju i preslikavaju u kompaktan opseg $[0, m - 1]$ (gde je m broj različitih X koordinata u skupu tačaka). Ovim se dubina segmentnog stabla strogo ograničava na $\lceil \log_2 m \rceil$ i minimizuje memorijska složenost.
2. **Brišuća prava:** Tačke se grupišu i sortiraju rastuće prema Y koordinatama. Brišuća prava se kreće odozdo nagore, zaustavljajući se samo na Y koordinatama gde se nalaze tačke. Svaka takva visina se posmatra kao diskretni trenutak u vremenu gde se prvo proveravaju potencijalni pravougaonici kojima je to gornja ivica, a zatim se tačke ubacuju u stablo i kreira se nova verzija segmentnog stabla.
3. **Perzistentno segmentno stablo:** Nad kompresovanim X koordinatama održava se broj tačaka na odgovarajućim pozicijama. Umesto kreiranja novog stabla iz početka za svaku visinu Y , koristi se perzistentnost zasnovana na deljenju čvorova. Pri dodavanju tačke na nekoj visini, alociraju se novi čvorovi isključivo duž putanje od korena do odgovarajućeg lista (njih $\mathcal{O}(\log m)$). Sva ostala podstabla koja ostaju neizmenjena dele se sa prethodnom verzijom pomoću pokazivača. Koren novonastale verzije stabla se pamti u mapi `rootAtY[y]` i time se istorija stanja čuva uz optimalan utrošak memorije.

3.1 Ispitivanje validnosti pravougaonika

Posmatraju se kandidati formirani od uzastopnih parova tačaka (x_1, x_2) na trenutnoj visini y , koji su se kao par već pojavili na nekoj ranijoj visini y_{bottom} . Validnost pravougaonika $[x_1, x_2] \times [y_{\text{bottom}}, y]$ se proverava u dva koraka:

- a) **Provera vertikalnih ivica** u složenosti $\mathcal{O}(1)$: Pomoću strukture `lastY_at_x` proverava se da li postoji međutačka na vertikalnim ivicama pravougaonika (poslednja viđena tačka za obe koordinate mora biti tačno y_{bottom}).

- b) **Provera unutrašnjosti pravougaonika** u složenosti $\mathcal{O}(\log m)$: Ako ivice ne sadrže međutačke, posmatra se opseg kolona strogo između x_1 i x_2 . Ako između njih uopšte nema drugih X koordinata u ravni, unutrašnjost je trivijalno prazna. U suprotnom, broj tačaka unutar pravougaonika dobija se oduzimanjem dva upita nad stablom: od broja tačaka u opsegu $[x_1, x_2]$ zabeleženih pre tekuće visine y , oduzima se broj tačaka zabeleženih na nivou donje ivice y_{bottom} u istom opsegu. Pravougaonik je validan ako i samo ako je ova razlika jednaka nuli.

4 Složenost

4.1 Vremenska složenost

Sortiranje tačaka i kompresija X koordinata zahtevaju $\mathcal{O}(N \log N)$ operacija.

Kako validan pravougaonik po definiciji problema ne sme sadržati tačke ni u unutrašnjosti ni na ivicama, na svakoj visini y dovoljno je posmatrati isključivo uzastopne parove tačaka (x_i, x_{i+1}) . Za k tačaka na datoj visini postoji $k - 1$ takvih susednih parova. Sumiranjem po svim diskretnim visinama dobija se da je ukupan broj razmatranih kandidata tokom rada algoritma strogo manji od N , odnosno najviše $\mathcal{O}(N)$.

Za svaki od najviše $\mathcal{O}(N)$ kandidata vrši se:

- provera vertikalnih ivica u konstantnom vremenu $\mathcal{O}(1)$,
- provera unutrašnjosti pomoću dva upita nad perzistentnim segmentnim stablom u vremenu $\mathcal{O}(\log m)$.

Ubacivanje svih N tačaka u perzistentno stablo traje $\mathcal{O}(N \log m)$, jer se pri svakom umećanju kreira najviše $\mathcal{O}(\log m)$ novih čvorova duž putanje do odgovarajućeg lista. Kako je broj jedinstvenih kolona $m \leq N$, ukupna vremenska složenost predloženog algoritma iznosi strogo $\mathcal{O}(N \log N)$, što predstavlja drastično ubrzanje u odnosu na naivni $\mathcal{O}(N^3)$ pristup.

4.2 Prostorna složenost

U analizi memorije posmatra se pomoćna (dodatna) prostorna složenost neophodna za rad samih algoritama, bez računanja prostora za skladištenje ulaznih tačaka:

- **Naivni algoritam** ima dodatnu prostornu složenost $\mathcal{O}(1)$, jer se ispitivanje vrši direktno nad ulaznim skupom tačaka uz nekoliko pomoćnih promenljivih i brojača.
- **Predloženi algoritam** pri svakom dodavanju tačke u stablo alocira se najviše $\mathcal{O}(\log m)$ novih čvorova, dok pomoćne strukture (`events`, `compX`, `lastY_at_x`, `rootAtY`, `seg`) zauzimaju linearan prostor $\mathcal{O}(N)$. Zbog toga ukupna dodatna prostorna složenost iznosi $\mathcal{O}(N \log m)$.

Algoritam	Vremenska složenost	Dodatna prostorna složenost
Naivni pristup	$\mathcal{O}(N^3)$	$\mathcal{O}(1)$
Brišuća prava + Perzistentno stablo	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log m)$

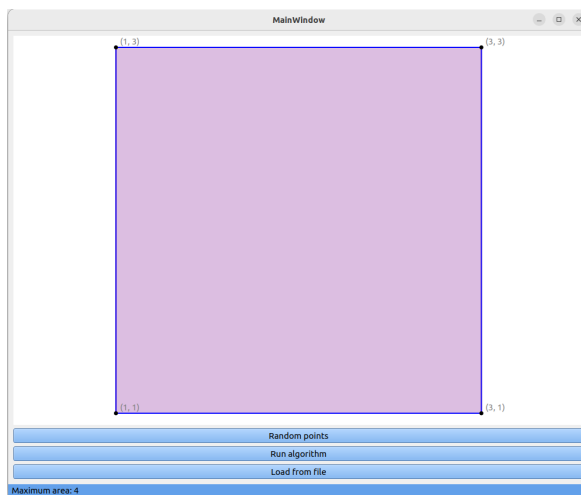
Tabela 1: Upporedni prikaz vremenske i dodatne prostorne složenosti ($m \leq N$ predstavlja broj jedinstvenih X koordinata).

5 Implementacija

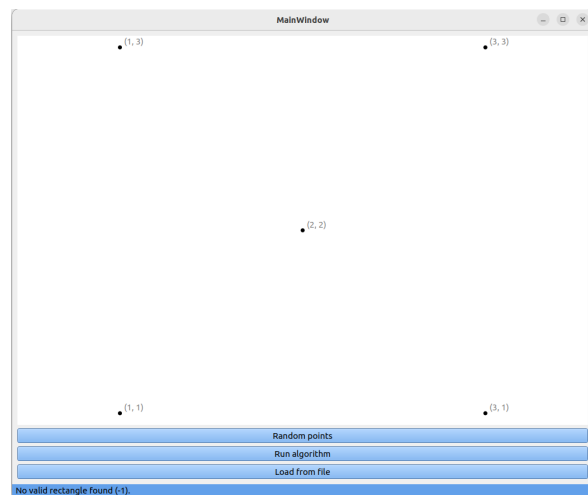
Projekat je implementiran u programskom jeziku C++ uz korišćenje biblioteke Qt za grafički korisnički interfejs i vizuelni prikaz rada algoritma. Korisnički interfejs omogućava:

- nasumično generisanje skupa tačaka,
- učitavanje koordinata iz tekstualne datoteke,
- vizuelno praćenje kretanja brišuće prave i isticanje pronađenih pravougaonika.

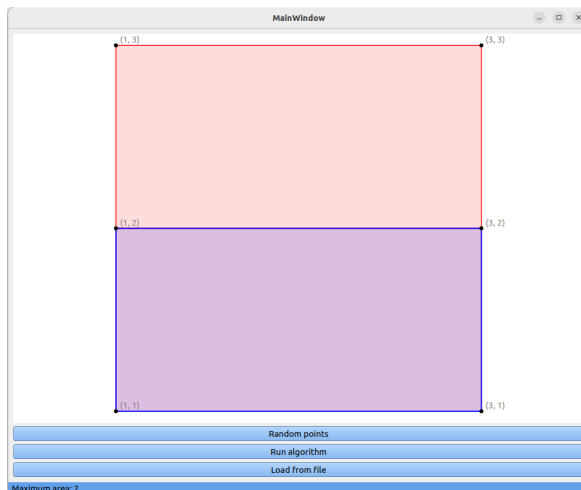
5.1 Primeri rada programa



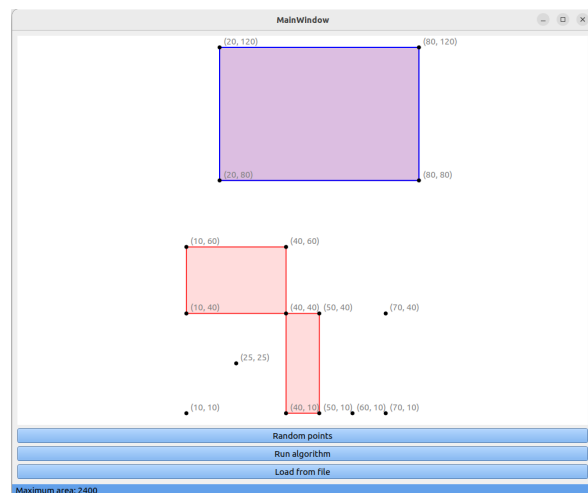
(a) Pronalazak pravougaonika površine 4



(b) Nema validnih pravougaonika (-1)



(c) Dva pravougaonika jednake površine (2)



(d) Tri pravougaonika, istaknut najveći (površine 2400)

Slika 1: Vizuelni prikaz rada algoritma kroz različite testne scenarije u Qt interfejsu.

Literatura

- [1] P. Janičić, *Geometrijski algoritmi*, Matematički fakultet, Univerzitet u Beogradu.
Dostupno na: <https://poincare.matf.bg.ac.rs/~predrag.janicic/courses/ga.pdf>

- [2] J. Marković, *Kurs Geometrijski algoritmi*, Matematički fakultet, Univerzitet u Beogradu.
Dostupno na: <https://faculty.matf.bg.ac.rs/p/jelena.markovic/kurs-geometrijski-algoritmi>
- [3] M. Jocović, *Master rad*, Matematički fakultet, Univerzitet u Beogradu.
Dostupno na: https://tezefiles.matf.bg.ac.rs/master_files/master_radovi/KRI/1079.2022/masterMirjanaJocovic.pdf
- [4] GeeksforGeeks, *Segment Tree Data Structure*.
Dostupno na: <https://www.geeksforgeeks.org/dsa/segment-tree-data-structure/>